MoverMate: архитектура кода

Сервис «переезд без хаоса»: план задач, учёт коробок и вещей, QR-этикетки, поиск, контроль погрузки и распаковки.

Хакатон PROD, кейс №1, команда 20. Документ описывает то, что реально написано и запущено на 17 октября 2026 (сверено с кодом и живым сайтом). Что не сделано, отмечено отдельно (раздел 12).

53	12	96 + 73	98%
эндпоинта API	таблиц в базе	тестов сервера и сайта	покрытие сервера

1. Что делает система и какие у неё ограничения

Пользователь создаёт проект переезда (название, дата, адреса необязательны), составляет план задач на три этапа (до, в день, после; типовые задачи — по кнопке), заводит коробки с комнатой назначения, быстро вписывает содержимое, печатает этикетки с QR-кодом, отмечает погрузку, доставку и распаковку коробок, ведёт статус каждой вещи (упакована, распакована, найдена, потеряна, повреждена) и ищет вещи по названию. Если вещь потерялась, система подсказывает, в каких нераспакованных коробках её искать. Помощники присоединяются к проекту по ссылке-приглашению.

Требование ТЗ	Как выполнено
Простота запуска, без внешних сервисов	Один Docker-образ, SQLite в томе; QR строится в браузере, внешних API нет
Номера коробок уникальны, повторный скан не создаёт дубль	Ограничение БД `UNIQUE(project_id, number)`, `GET` карточки ничего не создаёт
Статусы только по допустимым переходам	Таблица переходов в сервисе: допустим любой другой статус (решение владельца продукта), 409 на тот же статус и на пустую коробку дальше «собирается»
Одинаковые запросы дают одинаковые результаты	Поиск и списки с вторичными ключами сортировки (номер коробки, id)
Данные переживают обновление страницы	Данные в БД; список проектов приходит с сервера после входа, выбранный проект запомнен в браузере
Персональные данные необязательны, нет скрытых передач	Аккаунт — только логин и хэш пароля (scrypt), без почты и телефона; адреса и ответственный необязательны; нет внешних вызовов
Сотни коробок и тысячи вещей, пагинация	Индексы, `page` / `page_size`, нагрузочный проект на 300 коробок и 3000 вещей
Проект закрыт для посторонних	Вход по логину и паролю, проект видят только участники; помощники — по ссылке-приглашению; чужой проект отвечает 404
Журнал событий и проверка работоспособности	Таблица `events`, `GET /api/health` с запросом к БД; JSON-логи в формате Sage: каждый запрос, каждая ошибка с причиной и `request_id`, аудит ключевых действий; аппендер отправляет их в Sage

Задеплоено, CI/CD	Пайплайн GitLab: линтеры, тесты, образ, автодеплой на ВМ и проверка живого сайта после каждого слияния в `master`

2. Общая схема

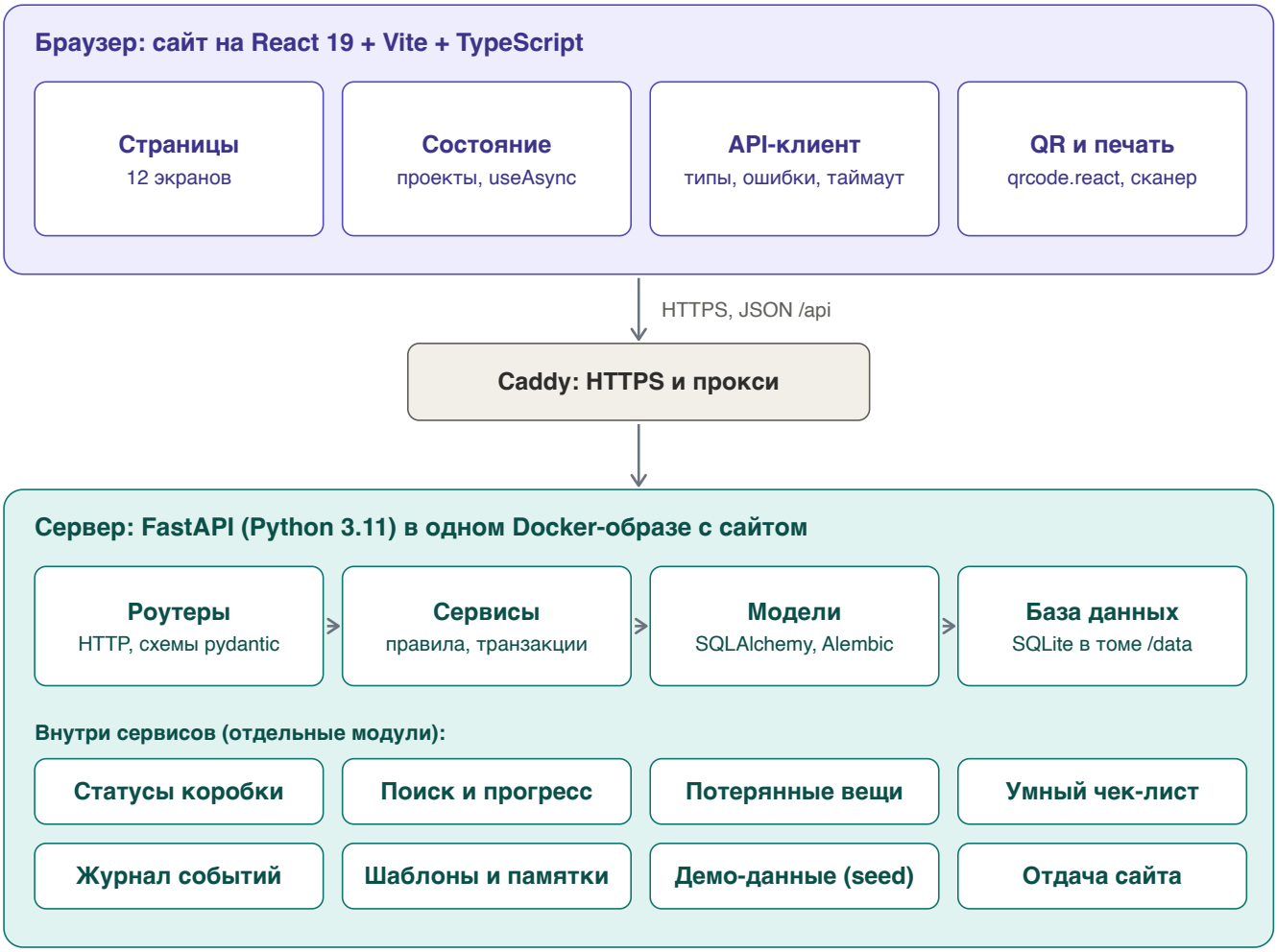


Рис. 1. Слои системы. Сайт и сервер едут в одном образе, база лежит в томе.

Сайт и API работают на одном origin: Caddy отдаёт сайт и проксирует `/api` на FastAPI, поэтому CORS не нужен. В режиме разработки запросы `/api` проксирует Vite (порт 5173 на порт 8000).

3. Сервер (backend)

Три слоя с чёткими обязанностями: роутер принимает и проверяет запрос, сервис содержит все правила и работает с транзакцией, модель описывает таблицы. Бизнес-правил в роутерах нет, поэтому сервисы можно тестировать без HTTP.

Модуль	Что внутри
`main.py`	Создание приложения, прослойка журнала запросов (request_id, время), обработчики ошибок, подключение роутеров, запуск справочников при старте

`routers/`	`auth` (вход, регистрация, демо, пароль, приглашения, участники), `projects` (проекты, задачи, события), `boxes` (коробки, вещи, поиск, этикетки, прогресс, первый день, потерянные, напоминания), `assistant`, `reference` (шаблоны, памятки), `health`, общий `confirm_param` для удаления
`services/boxes.py`	Самый крупный модуль (427 строк): выдача номера, машина статусов коробки, массовые переходы, вещи и их статусы, поиск и ранжирование, этикетки, прогресс
`services/auth.py`, `deps.py`	Пароли (scrypt), сессии и приглашения (в базе только SHA-256 токенов), ограничение неудачных входов; проверка участия в проекте в каждом запросе
`services/extras.py`	Чек-лист первого дня, потерянные вещи и подсказки, напоминания
`services/tasks.py`, `projects.py`	Задачи с сортировкой по этапу и сроку; создание проекта с шаблонными задачами (без сроков)
`services/assistant.py`	Умный чек-лист на локальных правилах, идемпотентное добавление
`services/events.py`	Запись события в той же транзакции, что и изменение
`models.py, db.py`	12 таблиц SQLAlchemy; движок с включёнными внешними ключами для SQLite
`schemas.py`	Контракт API на pydantic: ограничения длины, обрезка пробелов, пустая строка как «не заполнено»
`reference_data.py`, `seed.py`	Шаблоны задач, памятки, чек-лист первого дня, правила умного чек-листа; демо-проект и нагрузочный проект
`static_site.py`	Отдаёт собранный сайт и маршруты SPA (включая ссылки из QR), защищён от `../`; `/api/…` без маршрута остаётся честным 404

Пример: смена статуса коробки



Рис. 2. Один запрос сквозь слои. Блокировка и запись события в одной транзакции.

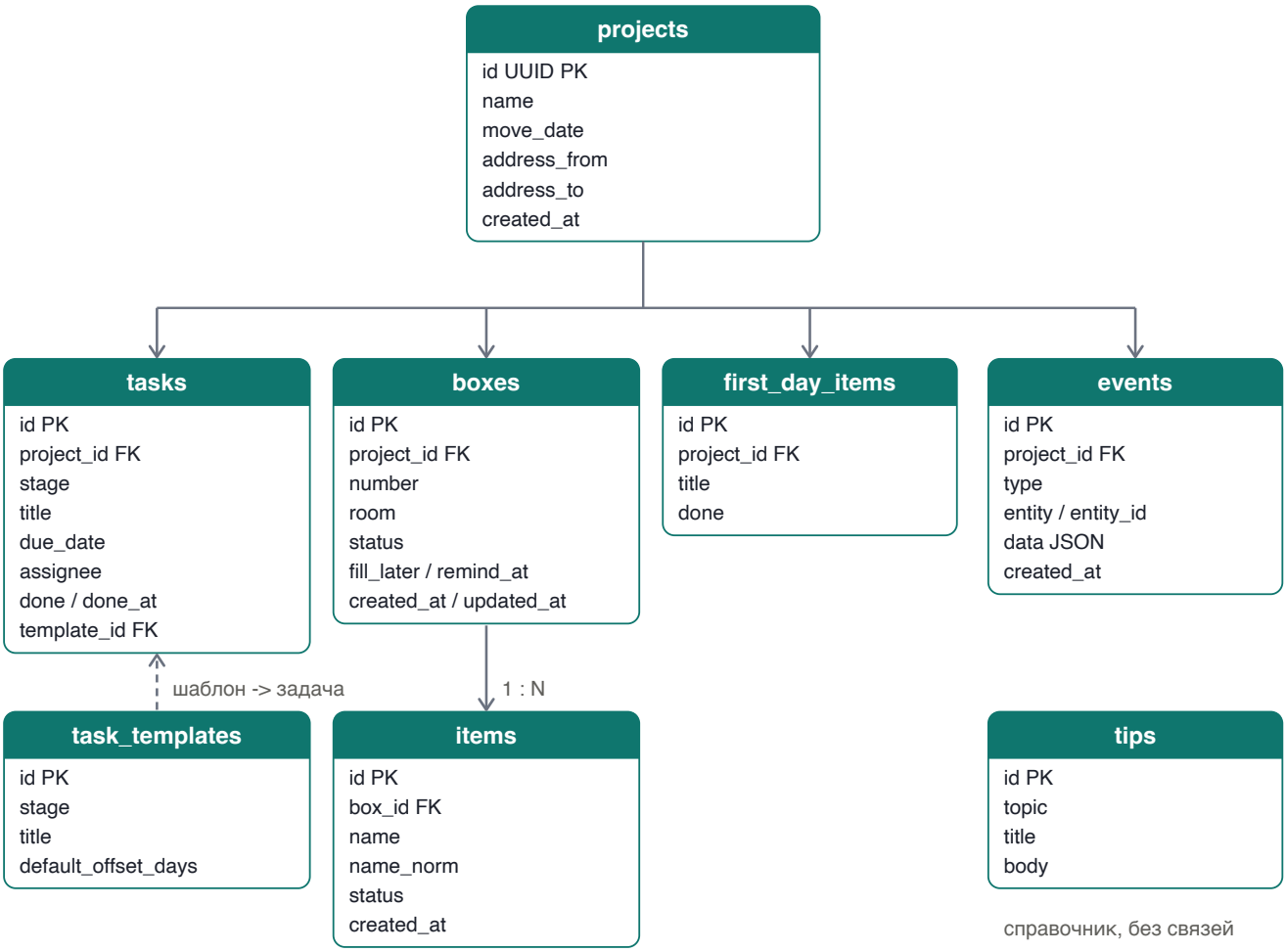
Единый формат ошибок

```
{"detail": "Нельзя перевести коробку из «packing» в «unpacked». Допустимо: packed",
 "code": "conflict", "request_id": "8a3232f8d2b9"}
```

Коды: 401 не вошли, 403 действие только для владельца, 404 нет сущности или проект чужой, 409 недопустимый переход, 429 перебор пароля, 422 неверные данные или нет `confirm=true` при удалении

(с перечнем полей в `errors`), 500 внутренняя ошибка (стек в логе, клиенту только `request_id`). Логи в формате JSON с идентификатором запроса.

4. Модель данных



Все дочерние таблицы удаляются каскадом вместе с проектом (ON DELETE CASCADE).

Ограничения: UNIQUE(project_id, number) у boxes; UNIQUE(project_id, template_id) у tasks (шаблон не дублируется).

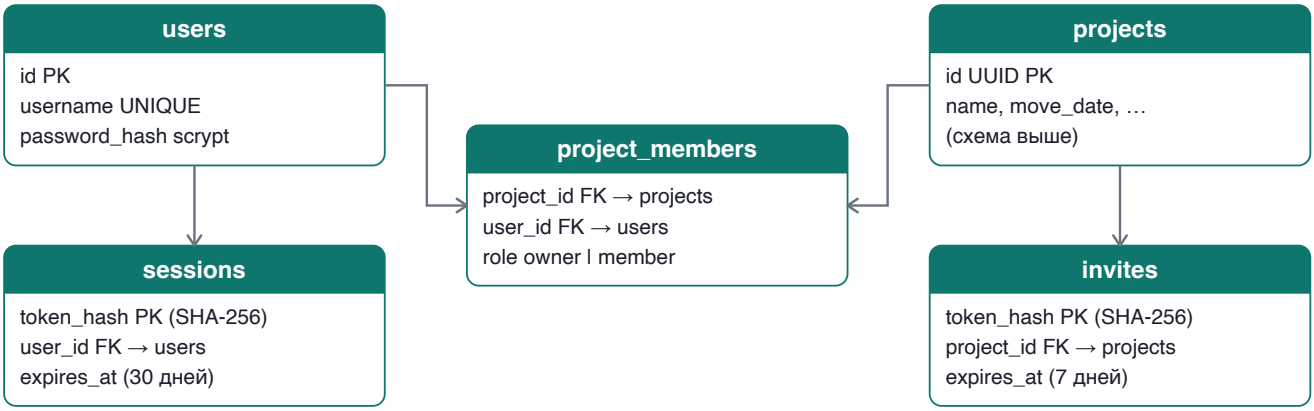
Рис. 3. Таблицы и связи. Миграции: `backend/alembic/versions/` — `0001` схема, `0002` время и категория задач, `0003` пользователи и доступ, `0004` статус вещи вместо флажка «потеряна», `0005` пометка «вещь первого дня».

- **Номер коробки** хранится отдельным числом, а не как `id`. Сервер выдаёт `max(number)+1` внутри проекта под блокировкой строки проекта; при гонке запись повторяется до трёх раз, а `UNIQUE` не даёт создать дубль. Ограничение: номер удалённой последней коробки будет выдан снова.
- **Статус вещи** — строка `packed | unpacked | found | lost | damaged`; в ответе API для совместимости есть и `lost = (status == "lost")`. Миграция `0004` перенесла старый флажок «потеряна» в статус и откатывается обратно.
- `name_norm` в вещах: имя без регистра, `ё = е`, схлопнутые пробелы. Поиск не зависит от функций СУБД и одинаково работает на SQLite и PostgreSQL.
- **Комната** это строка в коробке: у комнаты нет своих атрибутов, прогресс по комнатам считается группировкой.
- **Индексы:** `boxes(project_id, room, status)`, `items(box_id)`, `items(name_norm)`, `tasks(project_id, stage)`, `events(project_id, id)`, `project_members(project_id, user_id)` (уникальный, проверка

доступа в каждом запросе).

- **Справочники** (`task_templates`, `tips`) сидятся при каждом старте и добавляют только недостающее.

Доступ: пользователи, сессии, участники, приглашения



Стрелка — «один ко многим». В базе нет ни одного открытого пароля или токена: утечка БД не даёт войти и присоединиться к проекту.

5. Доменная логика: статусы коробки и вещи



Рис. 4. Машина статусов. Таблица переходов: `TRANSITIONS` в `services/boxes.py`.

Проверка переходов вынесена в чистую функцию `check_transition` и покрыта тестом на все 25 пар статусов (20 разрешены, 5 «в тот же» — 409). Смена статуса, запись в журнал и проверки идут в одной транзакции со `SELECT … FOR UPDATE` по коробке: две одновременные отметки идут по очереди. Массовая отметка комнаты (`bulk-transitions`) атомарна: одна недопустимая коробка отклоняет всю операцию.

На сайте статус меняется в окне «Сменить статус»: показаны все пять статусов с пояснением, текущий отмечен, выбрать можно любой другой.

Статус вещи

У каждой вещи свой статус: **упакована** (по умолчанию), **распакована**, **найдена**, **потеряна**, **повреждена**. В отличие от коробки, переходы свободные: вещь может найтись или оказаться повреждённой в любой момент. Статус выбирается в окне по клику на цветную метку вещи. Каждая смена пишется в журнал

(`item.lost`, `item.found` или `item.status_changed` с «из» и «в»); для потерянных вещей работают подсказки, где искать.

Правила, на которых держится сервис

Область	Правило
Поиск	Совпадения ранжируются: точное, с начала слова, вхождение; затем номер коробки и id. `%` и `_` в запросе обычные символы
Прогресс	«Погружено» в комнате = погружена + доставлена + распакована; «Распаковано» = распакованные / все коробки комнаты; охват описью = коробки с вещами / все
Напоминания	Коробка «заполнить позже», без вещей, срок не задан или наступил; вычисляются при запросе, без планировщика
Подсказки о потерянной вещи	Только нераспакованные коробки: где записана (1,0), где лежит похожее по основам слов (0,8), та же комната (0,6); не больше 10, порядок стабилен
Шаблоны задач	Новый проект создаётся с пустым планом; вариант «Обычный переезд» в «Умном чек-листе» добавляет 14 типовых задач без сроков и без дублей; там же своя особенность — название и задачи пользователя
Умный чек-лист	Правила для флагов «дети», «животные», «другой город» в `reference_data.py`; добавляются только отсутствующие по названию, без сроков; кнопка «Умный чек-лист» на «Плане», внешнего ИИ нет
Удаление	Всегда с `confirm=true`, иначе 422: случайный запрос не удаляет данные

6. API

Все 53 эндпоинта описаны в Swagger (`/api/docs`) и в `docs/API.md`. Вход по логину и паролю: сессия в httpOnly-cookie, проект видят только его участники (владелец и приглашённые по ссылке), чужой проект отвечает 404. Длинные списки идут страницами (`page`, `page_size` до 200).

Метод	Путь	Назначение
Служебные (4)		
GET	`/api/health`	Проверка работоспособности с запросом к БД
GET	`/api/config`	Ключ подсказок адреса (Google Maps), если задан
GET	`/api/task-templates`	Шаблоны задач
GET	`/api/tips`	Памятки (фильтр topic)
Вход (6)		
POST	`/api/auth/register`	Регистрация: логин и пароль, сразу вход
POST	`/api/auth/login`	Вход (429 при переборе)
POST	`/api/auth/demo`	Демо-вход для жюри без пароля
POST	`/api/auth/logout`	Выход
POST	`/api/auth/password`	Смена пароля, остальные входы закрываются
GET	`/api/auth/me`	Кто вошёл
Приглашения и участники (6)		
POST	`/api/projects/{pid}/invites`	Ссылка-приглашение на 7 дней (владелец)
DELETE	`/api/projects/{pid}/invites`	Отозвать все ссылки (владелец)
GET	`/api/invites/{token}`	Куда зовут (без входа)
POST	`/api/invites/{token}/accept`	Присоединиться к проекту
GET	`/api/projects/{pid}/members`	Участники проекта
DELETE	`/api/projects/{pid}/members/{user_id}?confirm=true`	Убрать участника или выйти самому
Проекты (6)		
GET	`/api/projects`	Мои проекты с ролью
POST	`/api/projects`	Создать проект: пустой план и чек-лист первого дня
GET	`/api/projects/{pid}`	Получить проект
PATCH	`/api/projects/{pid}`	Изменить название, дату, адреса
DELETE	`/api/projects/{pid}?confirm=true`	Удалить проект со всем содержимым
GET	`/api/projects/{pid}/events`	Журнал событий (страницы, фильтр type)
Задачи (5)		

GET	/api/projects/{pid}/tasks	Задачи (фильтры stage, done)
POST	/api/projects/{pid}/tasks	Добавить свою задачу
PATCH	/api/projects/{pid}/tasks/{tid}	Изменить или отметить выполнение
DELETE	/api/projects/{pid}/tasks/{tid}?confirm=true	Удалить задачу
POST	/api/projects/{pid}/tasks/apply-templates	Добавить типовые задачи (вариант «Обычный переезд» в умном чек-листе)
Коробки (9)		
GET	/api/projects/{pid}/boxes	Список коробок (room, status, fill_later, number, страницы)
POST	/api/projects/{pid}/boxes	Создать коробку, номер выдаёт сервер
GET	/api/projects/{pid}/boxes/{n}	Карточка коробки (её открывает QR)
PATCH	/api/projects/{pid}/boxes/{n}	Комната, «заполнить позже»
DELETE	/api/projects/{pid}/boxes/{n}?confirm=true	Удалить коробку с вещами
POST	/api/projects/{pid}/boxes/{n}/transitions	Сменить статус по допустимым переходам
POST	/api/projects/{pid}/boxes/bulk-transitions	Массовая смена статуса, всё или ничего
GET	/api/projects/{pid}/boxes/{n}/label	Данные этикетки (qr_path)
GET	/api/projects/{pid}/labels	Пакет этикеток (numbers=1,2,3, до 100)
Вещи (8)		
GET	/api/projects/{pid}/boxes/{n}/items	Вещи в коробке (страницы)
POST	/api/projects/{pid}/boxes/{n}/items	Добавить вещь
POST	/api/projects/{pid}/boxes/{n}/items/bulk	Быстрый ввод: строка = вещь, до 200
PATCH	/api/projects/{pid}/items/{id}	Переименовать, сменить статус вещи
DELETE	/api/projects/{pid}/items/{id}?confirm=true	Удалить вещь
GET	/api/projects/{pid}/items/lost	Потерянные вещи
GET	/api/projects/{pid}/items/first-day	Вещи первого дня
GET	/api/projects/{pid}/items/{id}/hints	Где искать потерянную вещь
Поиск и контроль (3)		
GET	/api/projects/{pid}/search?q=	Поиск вещи среди всех коробок
GET	/api/projects/{pid}/progress	Статусы и прогресс по комнатам
GET	/api/projects/{pid}/reminders	Коробки, которые пора описать
Первый день (4)		
GET	/api/projects/{pid}/first-day	Чек-лист первого дня
POST	/api/projects/{pid}/first-day	Добавить пункт
PATCH	/api/projects/{pid}/first-day/{id}	Отметить или переименовать
DELETE	/api/projects/{pid}/first-day/{id}?confirm=true	Удалить пункт

Умный чек-лист (2)		
POST	`/api/projects/{pid}/assistant/checklist`	Предложить задачи (дети, животные, другой город)
POST	`/api/projects/{pid}/assistant/checklist/apply`	Добавить предложенное, идемпотентно

7. Сайт (frontend)

React 19, TypeScript, Vite, `react-router-dom`, обычный CSS без UI-библиотек, иконки `lucide-react`, шрифт Inter. Зависимостей минимум, код короткий. Сканер QR (`html5-qrcode`) подгружается отдельным куском только при открытии экрана.

Маршруты и экраны

Путь	Экран
`/`	План: вкладки трёх этапов (новая задача — на открытый этап) и «Первый день» (вещи первого дня с коробкой, «+ Вещь», чек-лист); отметки срабатывают сразу
`/boxes`	Коробки: поиск вещи и номера, комнаты, напоминания
`/projects/:pid/boxes/:n`	Карточка (сюда ведёт QR)
… `/label`	Этикетка: номер, комната, QR
`/labels`, `/labels/print`	Список и печать всех этикеток
`/progress`	Счётчики, комнаты, чек-лист по распаковке вещей первого дня, потерянные вещи, памятка
`/scan`, `/new`	Сканер, новый проект
`/invite/:token`	Приглашение в проект
`/profile`	Личный кабинет: клик по проекту открывает его настройки (адреса, участники); сам проект открывается кнопкой с названием в шапке

Слои кода

Папка	Роль
`pages/`	Экраны, по одному файлу
`components/`	Каркас (`Shell`: меню, шапка с переключателем проекта, сканером и уведомлениями), окна (`Modal`, выбор статуса, уведомления), этикетка, формы
`state/`	Вход (`AuthContext`), проекты с сервера, `useAsync`, `useAction`, `useDebounced`
`api/`	Типы по контракту, клиент, функции эндпоинтов
`ui/`	Форматы, `StateView`, бейдж статуса

без входа	«О нас», «Вопросы» (частые вопросы), вход и регистрация (пароль дважды); разделы просят зарегистрироваться

Как работают экраны

- Каждый экран грузит данные через `useAsync` и показывает единые состояния «загрузка / ошибка с повтором / пусто» (`StateView`). Действия пользователя ловят ошибки сервера и показывают понятный текст.
- Клиент API (`api/client.ts`) ходит на `/api` того же origin, ставит таймаут 20 секунд, разбирает формат ошибок сервера и превращает ответ nginx в HTML (502) в понятное сообщение.
- Без входа любой адрес показывает экран входа, а адрес не меняется: после входа открывается та же страница (коробка по QR, приглашение). Список проектов приходит с сервера; в `localStorage` запомнен только выбранный проект, после выхода список стирается.
- **QR:** этикетка кодирует обычную ссылку `{origin}/projects/{pid}/boxes/{n}`; QR рисуется в браузере (`qrcode.react`). Любая камера телефона открывает её; на сайте есть и свой сканер, который принимает ссылку с любым адресом сайта.
- **Печать:** `window.print()` и правила `@media print` оставляют только этикетку; системный диалог печатает или сохраняет в PDF.
- **Уведомления:** колокольчик в шапке показывает напоминания об описи коробок и последние события журнала понятным текстом.
- **Адаптивность:** на телефоне вкладки снизу, на ноутбуке тёмная боковая панель; светлая и тёмная темы; цвет каждого статуса один на всех экранах.

8. Ключевые решения

Решение	Почему
Сайт вместо мобильного приложения	Симулятор и сборка APK съедали время при сроке в три дня; сайт проще проверять, а QR открывается любой камерой.
Вход по логину и паролю, участники и приглашения	Проект закрыт для посторонних, помощники приходят по ссылке на 7 дней. Храним только логин и хэш пароля (scrypt), токены сессий и приглашений — как SHA-256
Три слоя: роутер, сервис, модель	Правила не размазаны по контроллерам; сервисы тестируются без HTTP
Таблица переходов статусов	Новый статус или правило это одна строка плюс тест
Свободный выбор статуса коробки и вещи	На переезде отметки ставят с опозданием и пачками, пошаговая цепочка мешала (решение владельца продукта 17.10); защищаемся от пустой коробки и повторной отметки
Выбор статуса во всплывающем окне	Все варианты видны сразу с пояснением; одно окно для коробки, вещи и целой комнаты
Пустой план по умолчанию	Пользователь сам решает, нужны ли типовые задачи; кнопка добавляет их без дублей (ТЗ 5.1 — готовые шаблоны)

Типовые задачи без сроков	Сроки у всех разные; жёсткие даты от даты переезда чаще мешали, чем помогали
Номер в транзакции под блокировкой + UNIQUE	Уникальность гарантирует база, даже если сервис ошибётся
Один образ с сайтом и API	Проще деплой и шаблон CI организаторов (`Dockerfile` в корне); SQLite в томе: мало движущихся частей
Автодеплой по шаблону организаторов	Каждое слияние в `master` проверено и выкачено одинаково; перед выкаткой копия базы, после — проверка живого сайта
Caddy с автоматическим HTTPS	Камера сканера работает только по HTTPS; сертификат получается и продлевается сам
Умный чек-лист на правилах, без внешнего ИИ	ТЗ запрещает скрытые передачи данных; результат предсказуем и проверяем тестами

9. Развёртывание

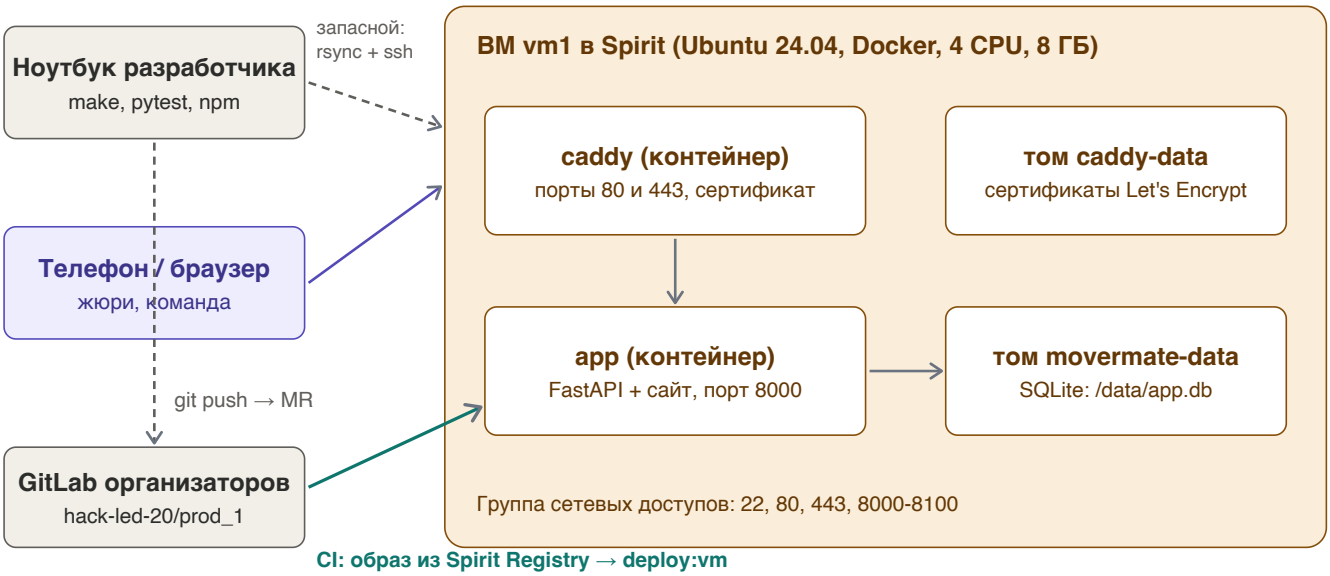
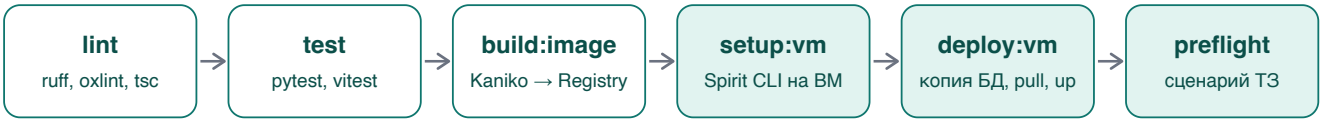


Рис. 5. Окружение. Основной путь — пайплайн GitLab; ручная выкатка осталась запасной.

- **Образ:** `Dockerfile` в корне, два этапа: сборка сайта (Node 22) и сервер (Python 3.11). При старте контейнер применяет миграции и запускает `uvicorn`; проверка живости `/api/health`.
- **Compose:** `deploy/docker-compose.yml` поднимает `app` и `caddy`; имя сервера для HTTPS задаётся в `deploy/.env` (`SITE_ADDRESS`).
- **Адрес:** https://hack-led-20-vm1-12902f.vmaas-spirit.t-education.com (сертификат Let's Encrypt).
- **Выкатка:** автоматически из пайплайна после слияния MR в `master` (раздел 10). Запасной путь — `rsync` исходников и `docker compose up -d --build` (`docs/DEPLOY_VM.md`).
- **Локально:** `scripts/run-site.sh` (сервер отдаёт API и сайт на порту 8000) или `make dev` + `npm run dev`.

10. CI/CD



белые — на каждой ветке и MR; зелёные — только после слияния в master

Рис. 6. Пайплайн `.gitlab-ci.yml` по шаблону организаторов `templates/l2sh-template`.

- **Процесс:** `master` защищён; изменения идут веткой и Merge Request. Пайплайн MR должен быть зелёным, после слияния пайплайн `master` сам выкатывает сайт.
- **Тесты в CI:** сервер на SQLite с отчётом покрытия (cobertura, процент виден в GitLab) и те же тесты на PostgreSQL — там проверяется параллельная выдача номеров коробок; миграции вверх-вниз-вверх; сайт — vitest и сборка.
- **Выкатка:** образ `registry.spirit.t-education.com/hack-led-20/prod_1:<sha>`; на ВМ копия базы в `/root/backups`, `docker compose pull app` и `up -d` в существующем стеке с Caddy, поэтому адрес и сертификат не меняются; `deploy/.env`, база и сертификаты не трогаются.
- **Проверка после выкатки:** `preflight:prod` проходит весь сценарий ТЗ на живом сайте через демо-вход. Если он красный, пайплайн красный.
- **Отличия от шаблона:** адреса `culab.ru` заменены на `t-education.com` (с раннеров не резолвятся), образ для ssh — `kroniak/ssh-client`, ключ `SSH_PRIVATE_KEY` очищается от `\r`.

11. Тестирование и проверка

Что	Сколько	Что проверяется
Сервер (pytest)	96 тестов, покрытие 98%	Правила статусов (все пары, пустая коробка), статусы вещей и журнал, шаблоны без сроков, самовосстановление демо, номера и дубли, поиск и порядок, прогресс, подсказки, напоминания, массовая погрузка, чек-лист и его идемпотентность, миграция совпадает с моделями, отдача сайта и защита от `../`
Сайт (vitest)	73 теста	Разбор ссылки из QR, форматы и склонения, клиент API (сеть, 409/422, 502), хранилище проектов (в том числе недоступное), состояния экрана, формы проекта, задачи и входа, окно выбора статуса, личный кабинет
Сквозная проверка	`scripts/preflight.sh`	Сайт и ссылка из QR, health, Swagger, демо-данные, полный сценарий ТЗ на временном проекте; запускается в CI после каждой выкатки

В локальном прогоне на SQLite пропущены 2 теста: параллельная выдача номеров (идёт в CI на PostgreSQL) и проверка аннотаций (только на Python 3.14). Нагрузка (300 коробок, 3000 вещей) замерялась вручную: списки, поиск и прогресс отвечают за единицы миллисекунд.

12. Что не сделано и ограничения

- **Не реализованы** план упаковки комнаты и опись коробки по фото из раздела 6 ТЗ (описаны в `docs/API.md` как развитие; опись по фото — только с явного согласия пользователя).
- **База — SQLite на одной ВМ.** Резервные копии: каждый час по cron и перед каждой выкаткой (`deploy/backup.sh`: онлайн-копия, проверка целостности, последние 48 копий). Отказоустойчивости второй ВМ нет.
- **Нет восстановления пароля**: почту не храним. Пароль можно сменить, зная текущий.
- **Номер удалённой последней коробки** выдаётся снова (номер считается как `max + 1`).
- **Напоминания** видны только в интерфейсе, без push и почты (осознанно: ТЗ не требует внешних каналов).
- **Камера сканера** требует HTTPS (на ВМ включён); на реальном телефоне проверена, есть ручной ввод номера.

13. Структура репозитория

```
backend/            сервер: app/ (код), alembic/ (5 миграций), tests/ (96 тестов)
frontend/           сайт: src/ (код, 73 теста), Dockerfile и nginx.conf (запуск без Caddy)
deploy/             docker-compose.yml, Caddyfile, .env.example для ВМ
.gitlab-ci.yml      пайплайн: lint → test → образ → деплой → preflight
Dockerfile          образ: сайт + сервер
docs/               CASE.md (минимум ТЗ), API.md, ARCHITECTURE.md, TEAM_PLAN.md, GITLAB.md,
DEPLOY_VM.md
scripts/            preflight.sh, run-site.sh, push-to-gitlab.sh, gitlab-login.sh
```